仿真 4：通信系统的“可靠性”与“有效性”

1. 实验目的

如下图所示，模拟灰度图像在 3 种通信系统中的传输效果。对比信源编码和信道编码对传输时效性和传输质量的影响，解释通信中“有效性”和“可靠性”的含义。

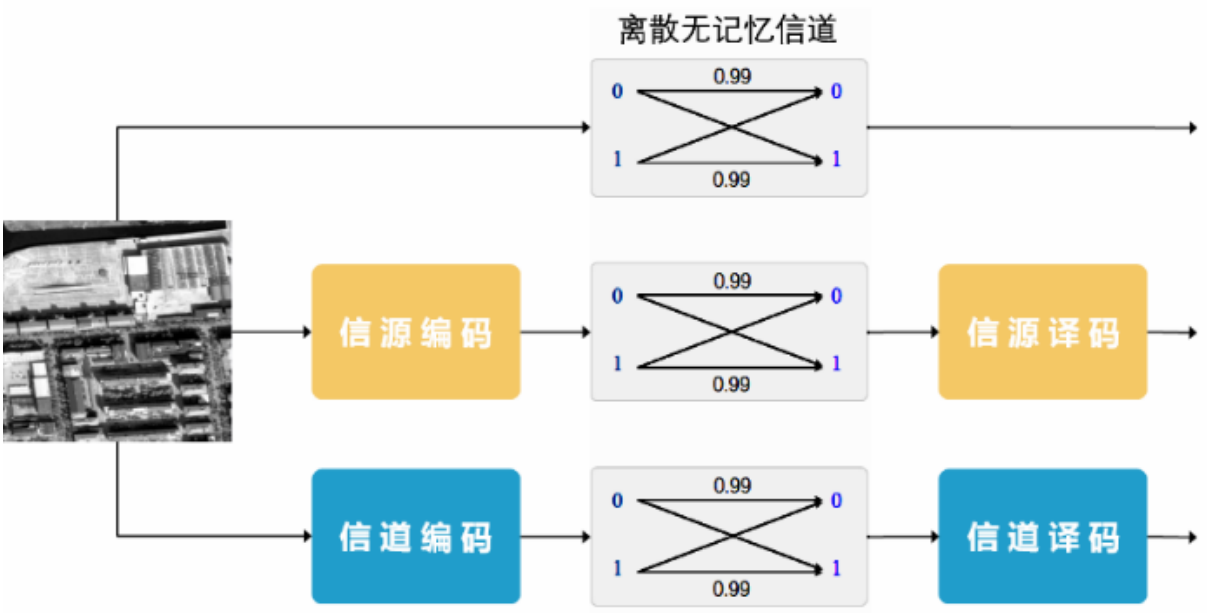


图 1: 实验框图

2. 实验原理

选择一张遥感灰度图像 gray_image.jpg ，在上图所示的 3 个通信系统中进行传输。

（1）在第一个系统中，以 8 位二进制数代表 256 阶灰度，将灰度图像转换为二元序列，通过二元对称信道进行传输。

（2）在第二个系统中，统计各阶灰度在图像中的概率分布，对各阶灰度进行二元霍夫曼编码，得到最佳代码组。依据该代码组，将灰度图像转换为二元序列，通过二元对称信道进行传输，并进行信源译码。

（3）在第三个系统中，以 8 位二进制数代表 256 阶灰度，将灰度图像转换为二元序列。对该序列进行信道编码（例如三次重复编码），通过二元对称信道进行传输，并进行信道译码。在上述 3 个通信系统中，二元对称

信道的信道矩阵均为:

$$P = \begin{bmatrix} p(0|0) = 0.99 & p(1|0) = 0.01 \\ p(0|1) = 0.01 & p(1|1) = 0.99 \end{bmatrix} \tag{1}$$

选择图像如下图所示：

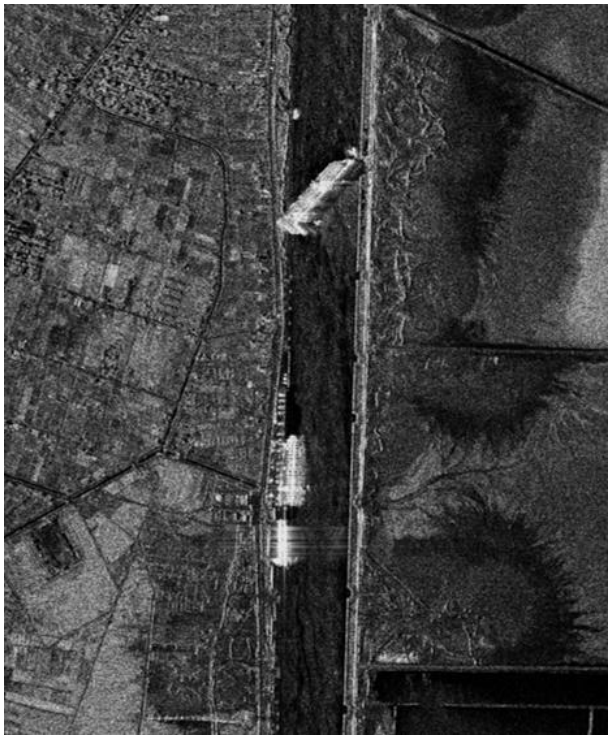


图 2: title-of-the-figure

对比上述通信系统输出图像的质量和信道输入序列的长度，从通信的“可靠性”“有效性”出发予以解释。

3. 实验过程

3.1 无信源编码和信道编码

将原始 (562, 464) 图像读入并转为长 562×464 的一维序列, 将每个 0-255 的灰度值根据 8421BCD 码转换为 8 位二进制数，得到长 562×464×8 的二元序列。将该序列通过二元对称信道进行传输，得到接收序列。再根据接受序列的 8 位二进制数转换为灰度值，得到输出图像。

3.2 有信源编码无信道编码

对原始图像进行灰度统计，得到各阶灰度的概率分布。根据该分布进行二元霍夫曼编码，得到最佳代码组。将原始图像转换为二元序列后，依

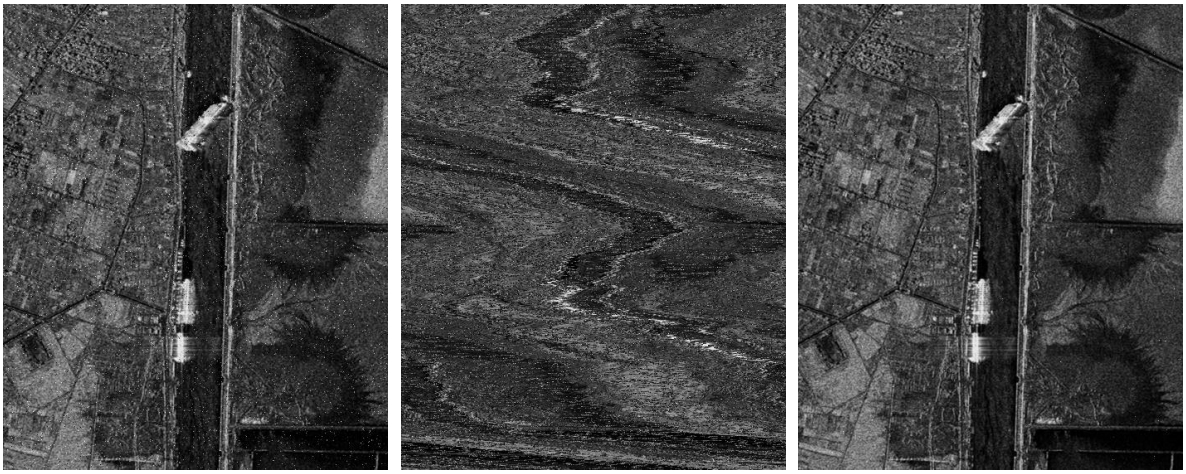
据最佳代码组进行信源编码，得到长 562×464×8 的二元序列。将该序列通过二元对称信道进行传输，得到接收序列。再根据接受序列的 8 位二进制数转换为灰度值，得到输出图像。

3.3 无信源编码有信道编码

在 8421BCD 编码后, 对二进制序列进行三次重复编码。将该序列通过二元对称信道进行传输，得到接收序列。再根据接受序列的 8 位二进制数转换为灰度值，得到输出图像。

结果展示

将上述三类通信系统的输出图像进行对比，结果如下所示：



(a) 定长编码　(b) 哈夫曼编码　(c) 信道重复编码

图 3: 三类通信系统输出图像对比

从展示结果上可以看出, 定长编码由于码字长度固定, 在信道中单个 bit 发生错误时, 会导致整个像素位置的灰度值错误, 从而导致图像失真, 但由于译码规则固定, 单个像素位值的错误不会影响到后续的译码环节。其噪声表现为鼓励个像素奇异值, 类似椒盐噪声。

而对于哈夫曼编码, 其码字长度不固定, 在单个像素发生译码错误后, 会导致后续的译码错误, 从而导致整个图像失真, 噪声非常严重, 因此图像质量远差于定长编码。

而信道重复编码由于对每个 bit 进行了三次重复, 在单个 bit 发生错误时, 可以通过多数表决的方式进行纠错, 最终噪声性能优于定长编码。

对三种图像计算 PSNR 指标，最终图像质量表现为信道重复编码 > 定

长编码 > 哈夫曼编码，符合分析结果。如下图所示:

```
定长编码 PSNR: 32.41 dB
哈夫曼编码 PSNR: 28.02 dB
信道重复编码 PSNR: 46.39 dB
```

图 4: 三类通信系统输出图像 PSNR 对比

4. 总结

从各类通信系统的编码特点，来分析三类通信系统的“可靠性”和“有效性”:

- 定长编码：由于码字长度固定，译码规则简单，在信道中单个 bit 发生错误时，会导致整个像素位置的灰度值错误，具有较高的可靠性。而定长编码未能充分考虑信源概率，因此在信息量上不够有效，导致传输效率较低。
- 哈夫曼编码：由于码字长度不固定，增加了单个符号携带的信息量，具有较高的有效性，但在信道中单个 bit 发生错误时，会导致后续的译码错误，降低了可靠性。
- 信道重复编码：通过对每个 bit 进行三次重复，可以提高可靠性，但由于增加了冗余，降低了有效性。

因此，在实际通信系统中，需要根据具体应用场景，综合考虑“可靠性”和“有效性”，选择合适的编码方式和信道编码策略，以达到最佳的通信效果。

5. 代码附录

```python
import heapq
import warnings
from PIL import Image
import os
import numpy as np
# 清楚 divide by zero 的警告
warnings.filterwarnings("ignore")

filename = __file__.split('\\')[-1].split('.')[0]

```

```python
class HuffmanNode:
    def __init__(self, char, freq):
        self.char = char
        self.freq = freq
        self.left = None
        self.right = None

    def __lt__(self, other):
        return self.freq < other.freq

def build_huffman_tree(probabilities):
    nodes = []
    for i, prob in enumerate(probabilities):
        node = HuffmanNode(i, prob)
        heapq.heappush(nodes, node)

    while len(nodes) > 1:
        left = heapq.heappop(nodes)
        right = heapq.heappop(nodes)
        merged = HuffmanNode(None, left.freq + right.freq)
        merged.left = left
        merged.right = right
        heapq.heappush(nodes, merged)

    return nodes[0]

def generate_huffman_codes(root):
    codes = {}

    def traverse(node, code):
        if node.char is not None:
            codes[node.char] = code
            return
        traverse(node.left, code + '0')
        traverse(node.right, code + '1')

    traverse(root, '')
    return codes

def huffman_encoding(probabilities):
    global root
    root = build_huffman_tree(probabilities)
    codes = generate_huffman_codes(root)
    codes = sorted(codes.items())
```

```
    sorted_values = [value for _, value in codes]
    return sorted_values

def jpg2npy(image_path):
    image = Image.open(image_path)
    array = np.array(image, dtype=np.uint8)
    return array

def npy2jpg(array, output_path):
    image = Image.fromarray(array.astype(np.uint8))
    image.save(output_path)

def channel_pass(input_: str):
    matirx = np.array([[0.99, 0.01], [0.01, 0.99]])
    return ''.join( list(map(lambda x: np.random.choice(['0', '1'],
        p=matirx[int(x)]), input_)))

def 定长编码(input_: np.ndarray) -> str:
    grey_img_binstr = ''.join(format(x, '08b') for x in input_)
    return grey_img_binstr

def 定长解码(input_: str) -> np.ndarray:
    grey_img = np.array([int(input_[i:i+8], 2) for i in range(0, len(input_), 8)],
        dtype=np.uint8)
    return grey_img

def 哈夫曼编码(input_: np.ndarray) -> str:
    probabilities = np.bincount(input_) / len(input_)
    huffman_codes = huffman_encoding(probabilities)
    grey_img_binstr = ''.join(huffman_codes[x] for x in input_)
    return grey_img_binstr

def 哈夫曼解码(input_: str) -> np.ndarray:
    global root
    decoded = []
    current_node = root
    for bit in input_:
        if bit == '0':
            current_node = current_node.left
        else:
            current_node = current_node.right
        if current_node.char is not None:
            decoded.append(current_node.char)
            current_node = root
```

```
    return np.array(decoded, dtype=np.uint8)

def 信道重复编码(input_: np.ndarray, n=3) -> str:
    grey_img_binstr = ''.join(format(x, '08b') for x in input_)
    repeated_bits = ''.join(bit * n for bit in grey_img_binstr)
    return repeated_bits

def 信道重复解码(input_: str, n=3) -> np.ndarray:
    decoded_bits = []
    for i in range(0, len(input_), n):
        bit_segment = input_[i:i+n]
        if bit_segment.count('1') > bit_segment.count('0'):
            decoded_bits.append('1')
        else:
            decoded_bits.append('0')
    decoded_bits = ''.join(decoded_bits)
    grey_img = np.array([int(decoded_bits[i:i+8], 2) \
        for i in range(0, len(decoded_bits), 8)], dtype=np.uint8)
    return grey_img

def exp4(encode=None, decode=None):
    grey_img = jpg2npy('gray_image.jpg')
    shape = grey_img.shape
    grey_img = grey_img.flatten()
    encoded = encode(grey_img)
    tmp = channel_pass(encoded)
    # tmp = encoded
    decoded = decode(tmp)
    if len( decoded ) != shape[0] * shape[1]:
        decoded = decoded[:shape[0] * shape[1]]
    elif len( decoded ) < shape[0] * shape[1]:
        decoded = np.pad(decoded, (0, shape[0] * shape[1] - len(decoded)),
             'constant')
    decoded = decoded.reshape(shape)
    npy2jpg(decoded, os.path.join(filename, f'{encode.__name__}.jpg'))


def psnr(original, compressed):
    max_pixel = 255.0
    mse = np.mean((original - compressed) ** 2)
    if mse == 0:
        return float('inf')
    psnr_value = 20 * np.log10(max_pixel / np.sqrt(mse))
    return psnr_value
```

```python

if __name__ == "__main__":
    if not os.path.exists(filename):
        os.mkdir(filename)
    exp4(encode=定长编码, decode=定长解码)
    exp4(encode=哈夫曼编码, decode=哈夫曼解码)
    exp4(encode=信道重复编码, decode=信道重复解码)
    grey_img = jpg2npy('gray_image.jpg')
    print(grey_img.shape)
    grey_1 = jpg2npy( os.path.join(filename, '定长编码.jpg') )
    grey_2 = jpg2npy( os.path.join(filename, '哈夫曼编码.jpg') )
    grey_3 = jpg2npy( os.path.join(filename, '信道重复编码.jpg') )
    print(f'定长编码 PSNR: {psnr(grey_img, grey_1):.2f} dB')
    print(f'哈夫曼编码 PSNR: {psnr(grey_img, grey_2):.2f} dB')
    print(f'信道重复编码 PSNR: {psnr(grey_img, grey_3):.2f} dB')
```